Generalizing Vs Specialising in Machine Learning

Ryan O'Neil

University Of Texas at Dallas

rco220002@utdallas.edu

Abstract— **This paper implements an LSTM from scratch in NumPy, verified with a numerical gradient check, to predict next-day movement for 12 S&P 500 stocks. On next-day returns the model cannot beat a mean baseline (skill 1.0002), while the same model predicting next-day volatility reaches 0.8691. Trained one stock at a time, the generalist slightly beats the Apple-only specialist (0.894 vs 0.914).**

Keywords— **LSTM, Skill, Gate, State**

I. Introduction

Why is financial data so hard to predict? There are a few reasons why, but I believe the most important ones are that prices increase, and fall based on new information and changes in the market. Once something starts working people will be insanely quick to copy and use until it is no longer usable. So, I narrowed down on an LSTM, also known as Long Short-Term Memory, simply because when predicting stocks, I believed that having access to previous information, or an accumulation of it, would help with recognising patterns and overall changes that happen over multiple days instead of treating everything independently. This choice was made before doing any research about which model is the best or why, because I wanted the experiment to prove a couple of things. The first was simply why it is hard to predict stocks and other financial data. The second was specialists versus generalists. After doing a bit of research I found that Gu, Kelly, and Xiu, covering the US cross-section over 30 years with 94 firm characteristics, report that "the R^2 oos is 0.33% for NN1 and peaks at 0.40% for NN3" [1]. Because this value is R^2 having a value that is low or closer to 0 is not always a failed experiment. Having a R^2 of .4% simply means that the model can explain under half of one percent of the variation. This is the best method we have to comprehensively compare these types of problems. My model which has a R^2 of -.30% should be compared to the .4 rather than something else. My model applied to next-day returns achieved a skill ratio of 1.0002. Skill is the models RMSE divided by the RSME of the baseline. The baseline being a model that always predicts the training set mean of the target. So a skill of 1.0 mean no improvement over the baseline and anything below 1.0 is better. Changing only the prediction target to next-day volatility, the same model achieved a skill of 0.8691. A model with a skill of 1.0002 is not a broken or underperforming model. Both have the same hyperparameter, however different targets, so they are simply a base for my overall question. Now diving into the second question, I wondered if a model that was trained on, or used a majority of, one stock would excel at only predicting that stock, and if so whether it would be better than a model that was trained on a bunch of different stocks. My pooled model with a skill of .8691 is computed using the collection of 12 stocks all at once training the model together. The per stock numbers in my forgetting matrix are computed against per stock training.

TABLE I
BASE MODELS SKILL

Model	Skill starting from Aapl
Trained on Apple Only	.914
After training through all 12 stocks (1 at a time)	.894

Now we can't exactly compare the numbers and instantly say generalist s better. But we can say that on Apple stocks generalist is slightly better than the specialist.

II. Theory

A. Problems With a RNN

A vanilla RNN or Recurrent Neural Network pushes its state through a squashing nonlinearity at every step. So, the gradient is multiplied by a factor less than 1 at each step [2], [3]. Now for my problem we must compute 60 days in a row, and if each step multiplies by around 0.5 then over 60

steps we get 0.5^{60} which is roughly 10^{-18}. The gradient reaching day one is gone before training even starts, so the network simply cannot learn anything that happened more than a few days back.

B. Fixing these problems with an LSTM

Now with an LSTM there are three gates that control what flows into and out of the states. The states are what hold information. There are forget, input, and output gates. Candidate or g is a proposed value or content the gates are acting on. There are two states, cell and hidden. Now the equations are as follows [3].

TABLE II
BASE MODELS SKILL

f_t = σ(W_f · [x_t, h_{t-1}] + b_f)	forget gate
i_t = σ(W_i · [x_t, h_{t-1}] + b_i)	input gate
g_t = tanh(W_g · [x_t, h_{t-1}] + b_g)	candidate
o_t = σ(W_o · [x_t, h_{t-1}] + b_o)	Output gate
c_t = f_t ⊙ c_{t-1} + i_t ⊙ g_t	Cell state
h_t = o_t ⊙ tanh(c_t)	Hidden state

Here σ is the sigmoid function, ⊙ is an elementwise multiply, and [x_t, h_{t-1}] simply means the input for that day and the hidden state from the day before concatenated into one vector.
Now for my code I only have W instead of W_f, W_i, W_g, and W_o because W is simply the 4 matrices stacked into one. We can simply use ConcatXandH @ W + b to compute the pre activations all at once and have the result split into the gate blocks. It is the same math as equations (1) to (4) just done in one matrix multiply instead of four. Since both f and i are independent nothing forces them to sum up to 1. So the cell can keep everything and write a lot at the same time, or it can freeze and change nothing at all. A GRU ties these two together and simply loses that freedom. Also, g uses tanh instead of sigmoid because the content or candidate needs a sign. Especially with stocks it must be able to rise and fall. Now the reasons the others are sigmoid and not tanh is simply because they are percentages of what stays or changes. You can't have -50% of something only 50%.

C. The Constant Error Carousel

The cell state has exactly one path from one step to the next and that is to be multiplied by f or the forget gate. This is simply the percentage of the content that is remembered or forgotten. The important part is that there is no squashing function anywhere in that path. In equation (5) c is only multiplied and added, it is never pushed through a tanh or a sigmoid. The squashing happens in equation (6) on the way out to h, not on the carry itself. So if f is close to 1 then the gradient passes backward basically unchanged, and that is the whole reason an LSTM can remember and a vanilla RNN cannot [2]. This is also why the forget gate bias is initialised to 1 rather than 0 [4]. If the bias is 0 the gate sits at σ(0) = 0.5, so the information is cut in half every single day, and 0.5^60 is about 10^-18 which is the exact same problem I described in part A. If the bias is 1 the gate starts at σ(1) which is about 0.73, and 0.73^60 is about 10^-8. That is ten orders of magnitude from one line of initialisation, and from there the model can learn to push the gate even higher if it needs to.

D. Backpropagation Through Time

The parameters are reused at every single timestep so their gradients are simply a sum of all T contributions accumulated by going through the data in reverse. State gradients pass backwards one step at a time. The cell state gradient is the exception because it receives gradient from two paths, h_t at the current step and c_{t+1} at the next step. The second path is what carries the long range gradient, and it is the same multiplication by f from part C only running backwards this time.

III. IMPLEMENTATION

A. Scope

In lstm.py all functions were created by me. All uses of libraries were for NumPy and for arithmetic. I used pandas for CSV parsing, requests for the download, and matplotlib for figures. I used no deep learning libraries anywhere.

B. Vectors

As stated, before I fused the four gate matrices into one at (D+H, 4H) array, so lstm.py produces all four pre activations. And implements loops over time only.

C. Sigmoid

From what I found NumPy has tanh but no sigmoid so I had to create one. I used exp(-|z|) so the exponential can never overflow regardless of the sign or magnitude of z.

D. Gradient Clipping

dW is a sum of 60 timesteps, so each contribution can occasionally cause the norm to blow up. This can cause the weights to go somewhere the training can never recover from. So I have code that resales when the global norm exceeds 5.

E. Verification

To make sure my backward pass was correct I checked every gradient against a central finite difference estimate. I would move one weight by +- 10^-6, run the forward pass twice, and compare the slope I measured against what my backward pass reported. Across all four parameter arrays the maximum relative error was 3.255 × 10^-7, so the analytical gradients match the numerical ones. I also ran the model on a synthetic task where I already knew there was something to learn, predicting the mean of a feature across the window. The same code took the loss from 0.0695 down to 0.0087 while the baseline sat at 0.0532, so about 6 times better than the baseline. I did both before running anything on real data. My main result is a negative one, so the first thing anyone would ask is whether my gradients were simply wrong. These two numbers answer that, because without them there is no way to tell the difference between finding no signal and having broken code.

F. Output layer Initialisation

At first I initialised the readout layer with the standard 1/sqrt(fan_in) like the rest of the weights. That gave me first predictions with a standard deviation of about 0.09 while my targets had a standard deviation of about 0.0125, so the model was off by a factor of seven before it even started. Most of the early training was simply spent shrinking the output instead of learning anything. So I scaled the readout weights by 0.02 at initialisation. That took the result from a skill of 1.37 after 150 epochs down to 1.01 after 30. Nothing about the architecture was wrong, it was one scalar. The reason this happens is that initialisation rules like 1/sqrt(fan_in) are built for hidden layers and they assume the targets are around unit scale. Daily returns are on the order of 10^-2, so the readout layer starts badly mis calibrated by default.

IV. Data and Experimental Setup

A. Dataset

I used 12 companies from Kagle S&P 500 dataset which has data from Feb 2013 to Feb 2018. That's 1259 trading days on each stock. I hosted it on GitHub where I uploaded the individual stock information for all the 12 stocks. My code downloads and caches the data automatically. Also, all stocks have identical calendars and an equal number of observations. Now there are 6 groups of companies Information Technology with AAPL, MSFT, and NVDA, Financials with JPM, GS, Energy with XOM, CVX, Health Care with JNJ, PFE, Consumer Staples with WMT, KO, and Industrials with only BA.

B. Features

"All methods agree on the same set of dominant predictive signals, a set that includes variations on momentum, liquidity, and volatility" [1]. Now my program has three features which map to each one of the predictive signals. Daily return = momentum, Volume change = liquidity, and Daily range = volatility. Price levels are never a feature because prices at most increase at around 1% a day. So if we used price as our match our answer would always fall in that 1% mark making our models "super accurate" but if we try to predict the price on from the year 2017 by training a model from 2013 it would be completely wrong and fail miserably.

C. Preprocessing

We have 4 main steps to preprocessing. That's build features and target from raw prices, split chronologically (the oldest 80% is the training data), fit the standardiser on the training days only but

apply to both, and build the windows inside each split separately.

D. Baselines

Regression Baseline Is predicting the training set mean of the target everyday. Skill = model RMSE / baseline RMSE. Anything below 1 beats the baseline. Now the directional baseline always predicts whichever side of the mean in more common. "As an aside, it is useful to know that there is a roughly 3% inflation in out-of-sample R2s if performance is benchmarked against historical averages. For OLS-3, the R2 relative to the historical mean forecast is 3.74% per month! Evidently, the historical mean is such a noisy forecaster that it is easily beaten by a fixed excess return forecasts of zero" [1]. Basically, they find the historical mean is so noisy that a fixed zero forecast beats it. Which inflates R^2 measured against it. My baseline is a single mean pooled over 11,352 observations rather than a per stock historical average, so it's far more stable that what they criticise. And when looking at volatility target the mean is nowhere near 0, so its unambiguously correct here as well.

E. Metrics

I used RMSE, MAE, skill on both R^2, correlation, directional accuracy against the baseline. While pred_std is a check on whether the model is varying its outputs or constantly guessing the more likely option.

V. Results

A. Predicting Next Day Retruns

My model on next day returns got an RMSE of 0.013248 while the baseline got 0.013245. That is a skill of 1.0002, which means there was no measurable improvement at all over simply predicting the training mean every single day. The R^2 was −0.003 so it was actually a very small amount worse than the mean, and the correlation between what I predicted and what happened was 0.0275 which is basically nothing. The directional accuracy was 0.4953 while the majority baseline was 0.5147. So the model called the direction correctly less often than simply always guessing whichever side was more common. It is important to compare against 0.5147 and not against 0.5, because the thing that has to be beaten is the trivial rule and not a coin flip. My model's directional calls are worth less than nothing. Now the most interesting number here is pred_std. My predictions had a standard deviation of 0.001097 while the real targets had a standard deviation of 0.013228. So my output only covers about 8% of the spread of the thing it is trying to predict. It has basically collapsed to guessing the mean over and over. I want to be clear that this is not the model failing to train. It is the model getting the right answer. If there is no signal to find then any move away from the mean simply increases the expected squared error, so predicting the mean is the correct solution. The model did exactly what it should have done. This lines up with what Gu, Kelly and Xiu found. The best neural network in the biggest study of this problem, using 94 firm characteristics over 30 years across the whole US market, only recovers under half a percent of monthly return variance [1]. My model with 3 features across 12 companies at a daily frequency recovers essentially none. So this is not me failing to clear a bar that everyone else cleared, it is roughly what the literature says should happen. All of this is consistent with the weak form of the efficient market hypothesis [5], which says that past prices on their own should not let you predict future prices. I want to be modest about that though. This is one architecture, one feature set, one horizon, and one five year window. It is consistent with the hypothesis, it does not prove it.

B. Predicting Next Day Volatility

When I changed the target to next day volatility the same model got an RMSE of 0.009717 against a baseline of 0.011181, so a skill of 0.8691. That is a 13% improvement. On MAE it did even better, 0.005628 against 0.007147, which is a skill of 0.7875 or a 21% improvement. The gap between those two numbers tells me the errors that are left are concentrated in a few extreme days, since MAE does not square the big misses the way RMSE does. The R^2 was 0.2326 and the correlation was 0.5183, so there is a real relationship here and not just a lucky run. The pred_std check tells the same story from the other side. It went from 0.001097 on returns up to 0.003645 here, so from 8% of the true spread up to 33%. The model is actually varying its

output now instead of hedging toward the mean. I want to point out that this positive result was predicted by theory rather than found by searching. Volatility clustering, meaning busy days follow busy days and quiet days follow quiet days, is one of the most reliable patterns in financial data and it is the exact thing ARCH and GARCH models were built for [6]. So a model with memory should do well here, and it did. One methodological note. I had to redefine directional accuracy for this target. Volatility is always positive, so comparing the sign of my prediction against zero gives a meaningless 100% every time. I found this as a bug in my early runs where direction came out as exactly 1.0. The metric now asks whether the model correctly called a busier or quieter than usual day compared to the training mean. On that measure it got 0.7491 against a majority baseline of 0.7232.

C. The Controlled Comparison

Everything about these two runs is identical. Same model, same features, same split, same hyperparameters, and same code. The only thing that changed is which column becomes the target, which is one line in data.py. And the skill moves from 1.0002 to 0.8691. That matters because it rules out the obvious objections. The null result on returns cannot be blamed on a broken implementation, or on the model not being big enough, or on it not training for long enough, because the exact same implementation at the exact same size trained for the exact same number of epochs finds real structure the moment I point it at a different target. Put together with the gradient check and the synthetic control from Section III, this is a measurement of how predictable the target is and not a measurement of whether my model works. So what is predictable is not the same thing as what you want to predict.

D. Hypermeters

I ran 11 configurations on each target, changing exactly one setting at a time away from the base run. These are shown in Table II and Figure 2. On the volatility target the best result came from changing the seed, which is not a hyperparameter at all. Changing only the seed moved skill by 0.0118. The best real hyperparameter change was the long window at 0.0062, which is about half as much. Changing the model size across a 4x range only spanned 0.0072. So most of that table is inside the run to run noise. My honest conclusion is that the model is insensitive to these hyperparameters over the ranges I tested. I am deliberately not naming a winner, because saying that I cannot separate them is the stronger and more correct claim. The one thing that might be real is that both learning rates away from 0.1 were 0.015 to 0.018 worse, which is right at the edge of the noise band. On the returns target nothing worked at all. All 11 configurations landed between 0.9962 and 1.0085, across a 4x range of model size, a 6x range of window length, and a 50x range of learning rate. This is a much stronger result than a single failed run, because nobody can ask me whether I tried a bigger model or trained it for longer. I did, and it is in the table.

The last thing I want to point out is the seed itself. Changing the seed moves skill by 0.0118 on volatility and only 0.0010 on returns, which is a factor of twelve. Initialisation only matters when there is something to find. So this is a completely separate piece of evidence pointing at the same conclusion as the controlled comparison above. The obvious fix for all of this is to run every configuration with several seeds and put error bars on the results, which is what I would do with more time.

TABLE IIII

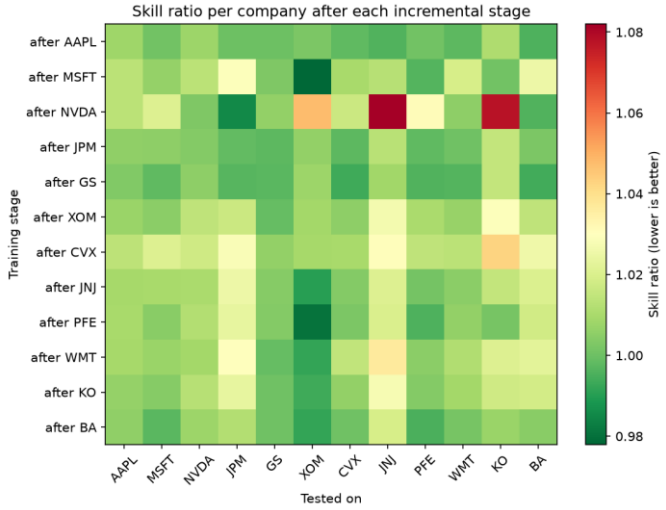
Run	Volatility skill	Returns skill
Base run	0.8691	1.0002
Smaller model	0.8765	0.9987
Bigger model	0.8693	0.9962
Short window	0.8664	1.0038
Long window	0.8629	1.0002
Small learning rate	0.8832	1.0016
Large learning rate	0.8873	1.0061
Fewer epochs	0.8681	1.0085
More epochs	0.8650	1.0056
Bigger batches	0.8726	0.9999
Different seed	0.8573	0.9992

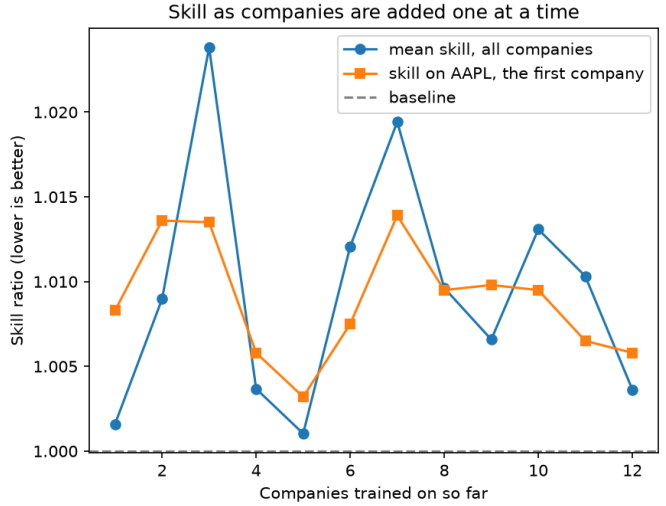
TABLE IV

Run	Volatility skill	Returns skill

Base run	0.8691	1.0002
Smaller model	0.8765	0.9987
Bigger model	0.8693	0.9962
Short window	0.8664	1.0038
Long window	0.8629	1.0002
Small learning rate	0.8832	1.0016
Large learning rate	0.8873	1.0061
Fewer epochs	0.8681	1.0085
More epochs	0.8650	1.0056
Bigger batches	0.8726	0.9999
Different seed	0.8573	0.9992

VI. Specialization Vs Generalization





I took one of my models and initilised it once and tgrained it through all the 12 companies one at a time with nothig reset between the stages. After each stage I tested it on each individual stocka dn comapred that to the model trained on the individual stocks. The generalists overall preformed better than the specialists. The confusing thing I found was that adding certain stocks actually made it worse like after adding NVDA some stocks actually preformed worse than before causing the model to have to catch up back where it was.

	AAPL	MSFT	NVDA	JPM	GS	XOM	CVX	JNJ	PFE	WMT	KO	BA
>> AAPL	1.008	1.001	1.008	1.000	1.000	1.003	0.998	0.996	1.001	0.998	1.011	0.996
>> MSFT	1.014	1.007	1.013	1.029	1.003	0.978	1.010	1.013	0.997	1.019	1.001	1.025
>> NVDA	1.014	1.021	1.003	0.986	1.006	1.048	1.017	1.082	1.031	1.006	1.078	0.996
>> JPM	1.006	1.006	1.004	0.998	0.998	1.006	0.998	1.013	0.998	1.000	1.015	1.003
>> GS	1.003	0.998	1.005	0.997	0.997	1.008	0.993	1.009	0.996	0.997	1.015	0.994
>> XOM	1.008	1.005	1.014	1.017	0.999	1.009	1.005	1.027	1.010	1.007	1.030	1.014
>> CVX	1.014	1.021	1.017	1.028	1.006	1.010	1.010	1.031	1.014	1.013	1.043	1.026
>> JNJ	1.010	1.010	1.010	1.025	1.004	0.990	1.004	1.021	1.001	1.005	1.015	1.020
>> PFE	1.010	1.005	1.012	1.024	1.004	0.981	1.003	1.020	0.996	1.006	1.002	1.018
>> WMT	1.010	1.007	1.009	1.030	0.999	0.992	1.014	1.037	1.005	1.011	1.021	1.022
>> KO	1.006	1.004	1.013	1.024	1.000	0.994	1.006	1.028	1.004	1.009	1.018	1.018
>> BA	1.006	0.997	1.008	1.012	1.001	0.992	1.000	1.019	0.995	1.002	1.007	1.004

References

[1] [1]S. Gu, B. Kelly, and D. Xiu, “Empirical Asset Pricing via Machine Learning,” The Review of Financial Studies, vol. 33, no. 5, pp. 2223–2273, Feb. 2020, doi: 10.1093/rfs/hhaa009.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997. doi: 10.1162/neco.1997.9.8.1735

[3] R. C. Staudemeyer and E. Rothstein Morris, "Understanding LSTM - a tutorial into Long Short-Term Memory Recurrent Neural Networks," arXiv:1909.09586, 2019.

[4] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," Neural Computation, vol. 12, no. 10, pp. 2451-2471, 2000. doi: 10.1162/089976600300015015

[5] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," The Journal of Finance, vol. 25, no. 2, pp. 383-417 May 1970. doi: 10.1111/j.1540-6261.1970.tb00518.x

[6] R. F. Engle, "Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation," Econometrica, vol. 50, no. 4, pp. 987-1007, Jul. 1982. doi: 10.2307/1912773

[7] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," Journal of Econometrics, vol. 31, no. 3, pp. 307-327, Apr. 1986. doi: 10.1016/0304-4076(86)90063-1